var_lock and the main thread sets the child thread to suspend. In this case, the main thread will wait indefinitely for *count_var_lock* and the child thread will be in suspend mode forever.

Strategies to avoid deadlock

There are deadlock prevention, deadlock avoidance and deadlock detection and recovery strategies. Deadlock prevention aims at removing one of the four conditions that are necessary for a deadlock to happen. Since 'mutual exclusion' is a condition that cannot be removed, we try to remove any one of the other three conditions. If we want to avoid the 'hold and wait conditions', then all the resources that a process/thread requires to complete its work inside the critical section must be acquired all at once. This leads to inefficient resource allocation and unnecessary holding of resources over long periods of time.

If we want to avoid the 'no preemption condition', then processes can be preempted of the resources they have. When resources are preempted, processes may lose whatever work they have done so far. This can lead to substantial overheads. If the 'circular wait' condition is to be avoided, then a programmer-imposed linear ordering of resources is enforced in the acquisition of resources. While 'acquire ordering of resources' is more efficient in terms of resource utilisation compared to the first two strategies, it is not flexible and requires a programmer to specify the ordering of resources for each system.

Consider our classic deadlock example where one thread acquires lock A first and then requests lock B, and a second thread acquires lock B first and then requests lock A. Such a condition will be avoided if a lock order is imposed in which lock A appears earlier than lock B. Hence, if the application code requires thread 2 to acquire lock B first and lock A next, then this will be a lock acquisition order violation. Such a violation can be flagged by a static source code analysis tool.

There are two types of deadlock prevention strategies, namely wait-die and wound-wait. In this case, each critical section is executed as a transaction. The transactions use a time-stamp to identify the older and younger transactions. Both in wait-die and in wound-wait schemes, a rolled back transaction is restarted with its original timestamp. Older transactions have precedence over newer ones, and starvation is hence avoided. The two schemes are as follows:

- In the **wait-die** scheme, an older transaction waits for a younger transaction to release a data item. Younger transactions never wait for older ones; they are rolled back instead. A transaction may die several times before acquiring the needed data item.
- In the **wound-wait** scheme, an older transaction forces rollback of younger transaction instead of waiting for it. Younger transactions may wait for older ones. There may be fewer rollbacks than in the *wait-die* scheme.

Given below is a small code snippet containing two concurrent transactions. Can you determine what will happen in a wait-die scheme and in a wound-wait scheme, respectively, for this code snippet?

```
T1                                  T2

Begin transaction                   Begin transaction
Read_item(customer 'A')
Write_item(customer 'A')

Read_item(order '123')              Read_item(customer 'A')
Write_item(order '123')
```

We have seen that deadlock and race conditions are two major issues a programmer has to keep in mind when writing multi-threaded code. There is another performance pitfall that many novice programmers fall into when writing multi-threaded code. It is known as false sharing, which we shall discuss next.

False sharing

Let's assume you have a single threaded application that is responsible for issuing movie tickets and counting how many were issued each day. And there is a global variable called *num_total_tickets* that gets incremented for each ticket issued. Here is the code snippet that does the increment:

```
int num_total_tickets = 0;

incrementTicketCounter()
{
        num_total_tickets++;
}
```

You were asked to parallelise the application and make it multi-threaded. You created N number of threads, each of which can do the ticket issue now. Therefore, the global variable *num_total_tickets* is a shared variable across all threads and needs to be updated by each thread when it issues a ticket. You used a mutex to protect the variable. Now your code looks like what's shown below:

```
int num_total_tickets = 0;
 pthread_mutex_t ticket_mutex;

incrementTicketCounter()
{
        pthread_mutex_lock(&ticket_mutex);
        num_total_tickets++;
        pthread_mutex_unlock(&ticket_mutex);
}
```

When you asked a colleague to review the code,